

Operator Overloading

- C# allows you to define the meaning of an operator relative to a class that you create.
- This process is called operator overloading.
- By overloading an operator, you expand its usage to your class.
- The effects of the operator are completely under your control and may differ from class to class.
- For example, a class that defines a linked list might use the + operator to add an object to the list.
- A class that implements a stack might use the + to push an object onto the stack.
- Another class might use the + operator in an entirely different way.

- When an operator is overloaded, none of its original meaning is lost.
- It is simply that a new operation, relative to a specific class, is added.
- Therefore, overloading the + to handle a linked list, for example, does not cause its meaning relative to integers (that is, addition) to be changed.
- A principal advantage of operator overloading is that it allows you to seamlessly integrate a new class type into your programming environment.
- This type extensibility is an important part of the power of an object-oriented language such as C#.
- Once operators are defined for a class, you can operate on objects of that class using the normal C# expression syntax.
- You can even use an object in expressions involving other types of data.
- Operator overloading is one of C#'s most powerful features.

Operator Overloading Fundamentals

- Operator overloading is closely related to method overloading.
- To overload an operator, use the operator keyword to define an operator method, which defines the action of the operator relative to its class.
- There are two forms of operator methods: one for unary operators and one for binary operators.

- The general form for each is shown here:
public static *ret-type operator op(param-type operand)*
{
// operations
}
// General form for overloading a binary operator
public static *ret-type operator op(param-type1
operand1, param-type1 operand2)*
{
// operations
}

- Here, the operator that you are overloading, such as + or /, is substituted for op.
- The ret-type specifies the type of value returned by the specified operation.
- Although it can be any type you choose, the return value is often of the same type as the class for which the operator is being overloaded.
- This correlation facilitates the use of the overloaded operator in expressions.

- For unary operators, the operand is passed in operand.
- For binary operators, the operands are passed in operand1 and operand2.
- Notice that operator methods must be both public and static.
- For unary operators, the operand must be of the same type as the class for which the operator is being defined.
- For binary operators, at least one of the operands must be of the same type as its class.

- Thus, you cannot overload any C# operators for objects that you have not created.
- For example, you can't redefine + for int or string.
- One other point: Operator parameters must not use the ref or out modifier.

Overloading Binary Operators

- To see how operator overloading works, let's start with an example that overloads two binary operators, the + and the –.
- The following program creates a class called ThreeD, which maintains the coordinates of an object in three-dimensional space.
- The overloaded + adds the individual coordinates of one ThreeD object to another.
- The overloaded – subtracts the coordinates of one object from the other.

```
// An example of operator overloading.
using System;
// A three-dimensional coordinate class.
class ThreeD {
int x, y, z; // 3-D coordinates
public ThreeD() { x = y = z = 0; }
public ThreeD(int i, int j, int k) { x = i; y = j; z = k; }
// Overload binary +.
public static ThreeD operator +(ThreeD op1, ThreeD op2)
{
ThreeD result = new ThreeD();
/* This adds together the coordinates of the two points
and returns the result. */
result.x = op1.x + op2.x; // These are integer additions
result.y = op1.y + op2.y; // and the + retains its original
result.z = op1.z + op2.z; // meaning relative to them.
return result;
}
```

```
// Overload binary -.
```

```
public static ThreeD operator -(ThreeD op1, ThreeD op2)  
{
```

```
    ThreeD result = new ThreeD();
```

```
    /* Notice the order of the operands. op1 is the left  
    operand and op2 is the right. */
```

- result.x = op1.x - op2.x; // these are integer subtractions
- result.y = op1.y - op2.y;
- result.z = op1.z - op2.z;
- return result;
- }
- // Show X, Y, Z coordinates.
- public void Show()
• {
• Console.WriteLine(x + ", " + y + ", " + z);
• }
• }

```
class ThreeDDemo {  
    static void Main() {  
        ThreeD a = new ThreeD(1, 2, 3);  
        ThreeD b = new ThreeD(10, 10, 10);  
        ThreeD c;  
        Console.Write("Here is a: ");  
        a.Show();  
        Console.WriteLine();  
        Console.Write("Here is b: ");  
        b.Show();  
        Console.WriteLine();  
        c = a + b; // add a and b together  
        Console.Write("Result of a + b: ");  
        c.Show();  
        Console.WriteLine();  
    }  
}
```

```
c = a + b + c; // add a, b, and c together
Console.Write("Result of a + b + c: ");
c.Show();
Console.WriteLine();
c = c - a; // subtract a
Console.Write("Result of c - a: ");
c.Show();
Console.WriteLine();
c = c - b; // subtract b
Console.Write("Result of c - b: ");
c.Show();
Console.WriteLine();
}
}
```

- This program produces the following output:

Here is a: 1, 2, 3

Here is b: 10, 10, 10

Result of $a + b$: 11, 12, 13

Result of $a + b + c$: 22, 24, 26

Result of $c - a$: 21, 22, 23

Result of $c - b$: 11, 12, 13

- Beginning with the overloaded operator **+**.
- **When two objects of type ThreeD are operated on by the + operator, the magnitudes of their respective coordinates are added together, as shown in `operator+( )`.**
- **Notice, however,** that this method does not modify the value of either operand.
- Instead, a new object of type **ThreeD**, which **contains the result of the operation**, is returned by the method.

- **To understand why the + operation does not change the contents of either object, think about the standard arithmetic + operation as applied like this: $10 + 12$.**
- **The outcome of this operation is 22, but** neither 10 nor 12 is changed by it.
- Although no rule prevents an overloaded operator from altering the value of one of its operands, it is best for the actions of an overloaded operator to be consistent with its usual meaning.

- Notice that **operator+()** returns an object of type **ThreeD**.
- **Although the method could** have returned any valid C# type, the fact that it returns a **ThreeD object allows the +** operator to be used in compound expressions, such as **a+b+c**.
- **Here, a+b generates a result** that is of type **ThreeD**.
- **This value can then be added to c.**
- **Had any other type of value been generated by a+b, such an expression would not work.**

- Here is another important point:
- When the coordinates are added together inside **operator+()**, the addition of the individual coordinates results in an integer addition.
- **This** is because the individual coordinates, **x**, **y**, and **z**, are integer quantities.
- **The fact that the + operator is overloaded for objects of type ThreeD has no effect on the + as it is applied to integer values**

- Now, look at **operator**–().
- **The – operator works just like the + operator except that the order of the parameters is important.**
- Recall that addition is commutative, but subtraction is not.
- (That is, $A - B$ is not the same as $B - A$)

- For all binary operators, the first parameter to an operator method will contain the left operand.
- The second parameter will contain the one on the right.
- When implementing overloaded versions of the noncommutative operators, you must remember which operand is on the left and which is on the right.

Overloading Unary Operators

- The unary operators are overloaded just like the binary operators.
- The main difference, is that there is only one operand.
- For example, here is a method that overloads the unary minus for the **ThreeD class**

```
// Overload unary -.  
public static ThreeD operator -(ThreeD op)  
{  
    ThreeD result = new ThreeD();  
    result.x = -op.x;  
    result.y = -op.y;  
    result.z = -op.z;  
    return result;  
}
```

- Here, a new object is created that contains the negated fields of the operand.
- This object is then returned.
- Notice that the operand is unchanged.
- Again, this is in keeping with the usual meaning of the unary minus.
- For example, in an expression such as ,
- $a = -b$
- **a receives the negation of b, but b is not changed.**

- In C#, overloading **++** and **--** is quite easy; simply return the incremented or decremented value, but don't change the invoking object.
- C# will automatically handle that for you,
- taking into account the difference between the prefix and postfix forms.

Handling Operations on C# Built-in Types

- For any given class and operator, an operator method can, itself, be overloaded.
- One of the most common reasons for this is to allow operations between a class type and other types of data, such as a built-in type.
- For example, consider the **ThreeD** class.
- **To this** point, you have seen how to overload the **+** so that it adds the coordinates of one **ThreeD** object to another.
- However, this is not the only way in which you might want to define addition for **ThreeD**.

- **For example, it might be useful to add an integer value to each coordinate of a ThreeD object.**
- **Such an operation could be used to translate axes.**
- **To perform such an operation, you will need to overload + a second time, as shown here:**

```
// Overload binary + for ThreeD + int.  
public static ThreeD operator +(ThreeD op1, int op2)  
{  
    ThreeD result = new ThreeD();  
    result.x = op1.x + op2;  
    result.y = op1.y + op2;  
    result.z = op1.z + op2;  
    return result;  
}
```

- Notice that the second parameter is of type **int**.
- **Thus, the preceding method allows an integer value to be added to each field of a ThreeD object.**
- **This is permissible because, as explained earlier, when overloading a binary operator, one of the operands must be of the same type as the class for which the operator is being overloaded.**
- However, the other operand can be of any other type.

Overloading the Relational Operators

- The relational operators, such as **=** or **<**, **can also be overloaded and the process is straightforward.**
- Usually, an overloaded relational operator returns a **true or false value.**
- This is in keeping with the normal usage of these operators and allows the overloaded relational operators to be used in conditional expressions.
- If you return a different type result, then you are greatly restricting the operator's utility.

- Here is a version of the **ThreeD** class that **overloads the < and > operators**.
- **In this** example, these operators compare **ThreeD objects based on their distance from the origin**.
- One object is greater than another if its distance from the origin is greater.
- One object is less than another if its distance from the origin is less than the other

- Given two points, such an implementation could be used to determine which point lies on the larger sphere.
- If neither operator returns true, then the two points lie on the same sphere.
- Of course, other ordering schemes are possible.

```
// Overload < and >.
```

```
using System;
```

```
// A three-dimensional coordinate class.
```

```
class ThreeD {
```

```
int x, y, z; // 3-D coordinates
```

```
public ThreeD() { x = y = z = 0; }
```

```
public ThreeD(int i, int j, int k) { x = i; y = j; z = k; }
```

// Overload <.

```
public static bool operator <(ThreeD op1, ThreeD  
    op2)
```

```
{
```

```
if(Math.Sqrt(op1.x * op1.x + op1.y * op1.y + op1.z *  
    op1.z) <
```

```
Math.Sqrt(op2.x * op2.x + op2.y * op2.y + op2.z *  
    op2.z))
```

```
return true;
```

```
else
```

```
return false;
```

```
}
```

```
// Overload >.
public static bool operator >(ThreeD op1, ThreeD
    op2)
{
    if(Math.Sqrt(op1.x * op1.x + op1.y * op1.y + op1.z *
        op1.z) >
        Math.Sqrt(op2.x * op2.x + op2.y * op2.y + op2.z *
            op2.z))
        return true;
    else
        return false;
}
```

```
// Show X, Y, Z coordinates.
```

```
public void Show()
```

```
{
```

```
    Console.WriteLine(x + ", " + y + ", " + z);
```

```
}
```

```
}
```

```
class ThreeDDemo {  
    static void Main() {  
        ThreeD a = new ThreeD(5, 6, 7);  
        ThreeD b = new ThreeD(10, 10, 10);  
        ThreeD c = new ThreeD(1, 2, 3);  
        ThreeD d = new ThreeD(6, 7, 5);  
        Console.WriteLine("Here is a: ");  
        a.Show();  
        Console.WriteLine("Here is b: ");  
        b.Show();  
        Console.WriteLine("Here is c: ");  
        c.Show();  
        Console.WriteLine("Here is d: ");  
        d.Show();  
    }  
}
```

```
Console.WriteLine();  
if(a > c) Console.WriteLine("a > c is true");  
if(a < c) Console.WriteLine("a < c is true");  
if(a > b) Console.WriteLine("a > b is true");  
if(a < b) Console.WriteLine("a < b is true");  
if(a > d) Console.WriteLine("a > d is true");  
else if(a < d) Console.WriteLine("a < d is true");  
else Console.WriteLine("a and d are same distance  
    from origin");  
}  
}
```

- The output from this program is shown here:

Here is a: 5, 6, 7

Here is b: 10, 10, 10

Here is c: 1, 2, 3

Here is d: 6, 7, 5

$a > c$ is true

$a < b$ is true

a and d are same distance from origin

- An important restriction applies to overloading the relational operators:
- You must overload them in pairs.
- For example, if you overload $<$, **you must also overload $>$, and vice versa.**

- The operator pairs are
- `==` and `!=`
- `<` and `>`
- `<=` and `>=`

Overloading true and false

- The keywords **true** and **false** can also be used as **unary operators for the purposes of** overloading.
- Overloaded versions of these operators provide custom determinations of true and false relative to classes that you create.
- Once true and false are overloaded for a class, you can use objects of that class to control the **if, while, for, and do-while statements**, or in a **? expression**.
- The **true and false operators must be overloaded as a pair**.
- **You cannot overload just one.**

- Both are unary operators and they have this general form:

```
public static bool operator true(param-type operand)
```

```
{
```

```
// return true or false
```

```
}
```

```
public static bool operator false(param-type  
operand)
```

```
{
```

```
// return true or false
```

```
}
```

- Notice that each returns a **bool result**.

Overloading the Logical Operators

- C# defines the following logical operators:
 - **&, |, !, &&, and ||.**
- **Of these, only the &, |, and ! can be overloaded.**
- **By following certain rules, however, the benefits of the short-circuit && and || can still be obtained.**

A Simple Approach to Overloading the Logical Operators

- Let's begin with the simplest situation.
- If you will not be making use of the short-circuit logical operators, then you can overload **&** and **|** as you would intuitively think, with each returning a **bool** result.
- An overloaded **!** will also usually return a **bool** result.

- Here is an example that overloads the **!**, **&**, and **|** **logical operators for objects of type ThreeD.**
- **As before, each assumes that a ThreeD object is true if at least one coordinate is non-zero.**
- If all three coordinates are zero, then the object is false.

// A simple way to overload !, |, and & for
ThreeD.

using System;

// A three-dimensional coordinate class.

class ThreeD {

int x, y, z; // 3-D coordinates

public ThreeD() { x = y = z = 0; }

public ThreeD(int i, int j, int k) { x = i; y = j; z = k;
}

```
// Overload |.
```

```
public static bool operator |(ThreeD op1,  
    ThreeD op2)
```

```
{
```

```
if( ((op1.x != 0) || (op1.y != 0) || (op1.z != 0)) |  
    ((op2.x != 0) || (op2.y != 0) || (op2.z != 0)) )
```

```
    return true;
```

```
else
```

```
    return false;
```

```
}
```

// Overload &.

```
public static bool operator &(ThreeD op1,  
    ThreeD op2)
```

```
{
```

```
if( ((op1.x != 0) && (op1.y != 0) && (op1.z != 0))  
    &
```

```
((op2.x != 0) && (op2.y != 0) && (op2.z != 0)) )
```

```
return true;
```

```
else
```

```
return false;
```

```
}
```

- `// Overload !.`
- `public static bool operator !(ThreeD op)`
- `{`
- `if((op.x != 0) || (op.y != 0) || (op.z != 0))`
- `return false;`
- `else return true;`
- `}`
- `// Show X, Y, Z coordinates.`
- `public void Show()`
- `{`
- `Console.WriteLine(x + ", " + y + ", " + z);`
- `}`
- `}`

```
class TrueFalseDemo {  
    static void Main() {  
        ThreeD a = new ThreeD(5, 6, 7);  
        ThreeD b = new ThreeD(10, 10, 10);  
        ThreeD c = new ThreeD(0, 0, 0);  
        Console.Write("Here is a: ");  
        a.Show();  
        Console.Write("Here is b: ");  
        b.Show();  
        Console.Write("Here is c: ");  
        c.Show();  
        Console.WriteLine();  
    }  
}
```

```
if(!a) Console.WriteLine("a is false.");  
if(!b) Console.WriteLine("b is false.");  
if(!c) Console.WriteLine("c is false.");  
Console.WriteLine();
```

```
if(a & b) Console.WriteLine("a & b is true.");  
else Console.WriteLine("a & b is false.");  
if(a & c) Console.WriteLine("a & c is true.");  
else Console.WriteLine("a & c is false.");  
if(a | b) Console.WriteLine("a | b is true.");  
else Console.WriteLine("a | b is false.");  
if(a | c) Console.WriteLine("a | c is true.");  
else Console.WriteLine("a | c is false.");  
}  
}
```

- The output from the program is shown here:

Here is a: 5, 6, 7

Here is b: 10, 10, 10

Here is c: 0, 0, 0

c is false.

a & b is true.

a & c is false.

a | b is true.

a | c is true.

Enabling the Short-Circuit Operators

- To enable the use of the **&&** and **||** short-circuit operators, you must follow four rules.
- First, the class must overload **&** and **|**.
- **Second, the return type of the overloaded & and | methods must be the same as the class for which the operators are being overloaded.**
- Third, each parameter must be a reference to an object of the class for which the operator is being overloaded.
- Fourth, the **true and false operators must be overloaded for the class.**

- **When** these conditions have been met, the short-circuit operators automatically become available for use.

- The following program shows how to properly implement the **& and |** for the **ThreeD** class so that the short-circuit operators **&& and ||** are available.

```
/* A better way to overload !, |, and & for ThreeD.  
This version automatically enables the && and ||  
operators. */  
using System;  
// A three-dimensional coordinate class.  
class ThreeD {  
    int x, y, z; // 3-D coordinates  
    public ThreeD() { x = y = z = 0; }  
    public ThreeD(int i, int j, int k) { x = i; y = j; z = k; }
```

// Overload | for short-circuit evaluation.

```
public static ThreeD operator |(ThreeD op1,  
    ThreeD op2)
```

```
{
```

```
if( ((op1.x != 0) || (op1.y != 0) || (op1.z != 0)) |  
    ((op2.x != 0) || (op2.y != 0) || (op2.z != 0)) )
```

```
    return new ThreeD(1, 1, 1);
```

```
else
```

```
    return new ThreeD(0, 0, 0);
```

```
}
```

// Overload & for short-circuit evaluation.

```
public static ThreeD operator &(ThreeD op1,  
    ThreeD op2)
```

```
{
```

```
if( ((op1.x != 0) && (op1.y != 0) && (op1.z != 0))  
    &
```

```
((op2.x != 0) && (op2.y != 0) && (op2.z != 0)) )
```

```
return new ThreeD(1, 1, 1);
```

```
else
```

```
return new ThreeD(0, 0, 0);
```

```
}
```

// Overload !.

```
public static bool operator !(ThreeD op)
{
    if(op) return false;
    else return true;
}
```

// Overload true.

```
public static bool operator true(ThreeD op) {  
    if((op.x != 0) || (op.y != 0) || (op.z != 0))  
        return true; // at least one coordinate is  
            non-zero  
    else  
        return false;  
}
```

- `// Overload false.`
- `public static bool operator false(ThreeD op) {`
- `if((op.x == 0) && (op.y == 0) && (op.z == 0))`
- `return true; // all coordinates are zero`
- `else`
- `return false;`
- `}`

```
// Show X, Y, Z coordinates.
```

```
public void Show()
```

```
{
```

```
    Console.WriteLine(x + ", " + y + ", " + z);
```

```
}
```

```
}
```

```
class TrueFalseDemo {  
    static void Main() {  
        ThreeD a = new ThreeD(5, 6, 7);  
        ThreeD b = new ThreeD(10, 10, 10);  
        ThreeD c = new ThreeD(0, 0, 0);  
        Console.Write("Here is a: ");  
        a.Show();  
        Console.Write("Here is b: ");  
        b.Show();  
        Console.Write("Here is c: ");  
        c.Show();  
        Console.WriteLine();  
    }  
}
```

```
if(a) Console.WriteLine("a is true.");  
if(b) Console.WriteLine("b is true.");  
if(c) Console.WriteLine("c is true.");  
if(!a) Console.WriteLine("a is false.");  
if(!b) Console.WriteLine("b is false.");  
if(!c) Console.WriteLine("c is false.");  
Console.WriteLine();
```

```
Console.WriteLine("Use & and |");  
if(a & b) Console.WriteLine("a & b is true.");  
else Console.WriteLine("a & b is false.");  
if(a & c) Console.WriteLine("a & c is true.");  
else Console.WriteLine("a & c is false.");  
if(a | b) Console.WriteLine("a | b is true.");  
else Console.WriteLine("a | b is false.");  
if(a | c) Console.WriteLine("a | c is true.");  
else Console.WriteLine("a | c is false.");  
Console.WriteLine();
```

// Now use short-circuit ops.

```
Console.WriteLine("Use short-circuit && and ||");  
if(a && b) Console.WriteLine("a && b is true.");  
else Console.WriteLine("a && b is false.");  
if(a && c) Console.WriteLine("a && c is true.");  
else Console.WriteLine("a && c is false.");  
if(a || b) Console.WriteLine("a || b is true.");  
else Console.WriteLine("a || b is false.");  
if(a || c) Console.WriteLine("a || c is true.");  
else Console.WriteLine("a || c is false.");  
}  
}
```

- The output from the program is shown here:

Here is a: 5, 6, 7

Here is b: 10, 10, 10

Here is c: 0, 0, 0

a is true.

b is true.

c is false.

Use & and |

a & b is true.

a & c is false.

a | b is true.

a | c is true.

Use short-circuit && and ||

a && b is true.

a && c is false.

a || b is true.

a || c is true.

- Let's look closely at how the **&** and **|** are implemented. They are shown here:

// Overload | for short-circuit evaluation.

```
public static ThreeD operator |(ThreeD op1, ThreeD op2)
{
    if( ((op1.x != 0) || (op1.y != 0) || (op1.z != 0)) |
        ((op2.x != 0) || (op2.y != 0) || (op2.z != 0)) )
        return new ThreeD(1, 1, 1);
    else
        return new ThreeD(0, 0, 0);
}
```

// Overload & for short-circuit evaluation.

```
public static ThreeD operator &(ThreeD op1,  
    ThreeD op2)
```

```
{
```

```
if( ((op1.x != 0) && (op1.y != 0) && (op1.z != 0))  
    &
```

```
((op2.x != 0) && (op2.y != 0) && (op2.z != 0)) )
```

```
return new ThreeD(1, 1, 1);
```

```
else
```

```
return new ThreeD(0, 0, 0);
```

```
}
```

- Notice first that both now return an object of type **ThreeD**.
- **Pay attention to how this object** is generated.
- If the outcome of the operation is true, then a true **ThreeD object (one in which** at least one coordinate is non-zero) is created and returned.
- If the outcome is false, then a false object is created and returned.

- Thus, in a statement like this
if(a & b) Console.WriteLine("a & b is true.");
else Console.WriteLine("a & b is false.");
- the outcome of a & b is a ThreeD object, which in this case is a true object.
- Since the operators true and false are defined, this resulting object is subjected to the true operator, and a bool result is returned.
- In this case, the result is true and the **if** succeeds.

- Because the necessary rules have been followed, the short-circuit operators are now available for use on **ThreeD objects**.
- **They work like this.**
 - The first operand is tested by using operator true (for `||`) or operator false (for `&&`).
 - If it can determine the outcome of the operation, then the corresponding `&` or `|` is not evaluated.
 - Otherwise, the corresponding overloaded `&` or `|` is used to determine the result.
- Thus, using a `&&` or `||` causes the corresponding `&` or `|` to be invoked only when the first operand cannot determine the outcome of the expression.

- For example, consider this statement from the program:

```
if(a || c) Console.WriteLine("a || c is true.");
```

- The **true operator** is first applied to a.
- **Since a is true in this situation, there is no need to use the | operator method.**

- **However, if the statement were rewritten like this:**

```
if(c || a) Console.WriteLine("c || a is true.");
```

- **then the `true` operator would first be applied to `c`, which in this case is false.**
- **Thus, the `|` operator method would be invoked to determine if `a` was true (which it is in this case).**

- There are some restrictions to overloading operators.
- We cannot alter the precedence of any operator.
- We cannot alter the number of operands required by the operator, although the operator method could choose to ignore an operand.
- There are several operators that we cannot overload.
- Perhaps most significantly, we cannot overload any assignment operator, including the compound assignments, such as `+=`.

Other operators that cannot be overloaded

&&	()	.	?
??	[]		=
=>	→	as	checked
default	is	new	sizeof
typeof	unchecked		

Conversion Operators

- To use an object of a class in an expression involving other types of data.
- Or a simple type conversion from the class type to the target type.

- C# allows you to create a special type of **operator**
- method called a *conversion operator*.
- *A conversion operator converts an object of one class into another type.*
- There are two forms of conversion operators, **implicit and explicit**.

The general form:

- public static operator implicit
target-type(source-type v) { return value; }
- public static operator explicit
target-type(source-type v) { return value; }

- *target-type* is the target type that are converted to;
- *source-type* is the type are converted from;
- *value* is the value of the class after conversion.
- *The conversion* operators return data of type *target-type*, and no other return type specifier is allowed.

- If the conversion operator specifies **implicit**, then **the conversion is invoked automatically**, such as when an object is used in an expression with the target type.
- When the conversion operator specifies **explicit**, **the conversion is invoked when a cast is used**.
- Both an implicit and explicit conversion operator cannot be defined for the same target and source types.

- To illustrate a conversion operator, we will create one for the **ThreeD class**.
- **Suppose we** want to convert an object of type **ThreeD into an integer so it can be used in an integer** expression.
- Further, the conversion will take place by using the product of the three dimensions.
- To accomplish this, we will use an implicit conversion operator that looks like this:

```
public static implicit operator int(ThreeD op1)
{
    return op1.x * op1.y * op1.z;
}
```

```
// An example that uses an implicit conversion operator.  
using System;  
// A three-dimensional coordinate class.  
class ThreeD {  
    int x, y, z; // 3-D coordinates  
    public ThreeD() { x = y = z = 0; }  
    public ThreeD(int i, int j, int k) { x = i; y = j; z = k; }  
    // Overload binary +.  
    public static ThreeD operator +(ThreeD op1, ThreeD op2)  
    {  
        ThreeD result = new ThreeD();  
        result.x = op1.x + op2.x;  
        result.y = op1.y + op2.y;  
        result.z = op1.z + op2.z;  
        return result;  
    }  
}
```

```
// An implicit conversion from ThreeD to int.  
public static implicit operator int(ThreeD op1)  
{  
    return op1.x * op1.y * op1.z;  
}  
  
// Show X, Y, Z coordinates.  
public void Show()  
{  
    Console.WriteLine(x + ", " + y + ", " + z);  
}  
}
```

```
class ThreeDDemo {  
    static void Main() {  
        ThreeD a = new ThreeD(1, 2, 3);  
        ThreeD b = new ThreeD(10, 10, 10);  
        ThreeD c = new ThreeD();  
        int i;  
        Console.Write("Here is a: ");  
        a.Show();  
        Console.WriteLine();  
        Console.Write("Here is b: ");  
        b.Show();  
        Console.WriteLine();  
    }  
}
```

```
c = a + b; // add a and b together
Console.WriteLine("Result of a + b: ");
c.Show();
Console.WriteLine();
i = a; // convert to int
Console.WriteLine("Result of i = a: " + i);
Console.WriteLine();
i = a * 2 - b; // convert to int
Console.WriteLine("result of a * 2 - b: " + i);
}
}
```

This program displays the output:

Here is a: 1, 2, 3

Here is b: 10, 10, 10

Result of $a + b$: 11, 12, 13

Result of $i = a$: 6

result of $a * 2 - b$: -988

- As the program illustrates, when a **ThreeD** object is used in an integer expression, such as `i = a`, the conversion is applied to the object.
- In this specific case, the conversion returns the value 6, which is the product of coordinates stored in **a**.
- However, when an expression does not require a conversion to **int**, the conversion operator is not called.
- This is why `c = a + b` *does not invoke **operator int()***.

- We can create different conversion operators to meet different needs.
- We could define a second conversion operator that converts **ThreeD to double, for example.**
- Each conversion is applied automatically and independently.
- An implicit conversion operator is applied automatically when a conversion is required in an expression, when passing an object to a method, in an assignment, and also when an explicit cast to the target type is used.

- Alternatively, you can create an explicit conversion operator, which is invoked only when an explicit cast is used.
- An explicit conversion operator is not invoked automatically.
- For example, here is the previous program reworked to use an explicit conversion to **int**:

// Use an explicit conversion.

using System;

// A three-dimensional coordinate class.

class ThreeD {

int x, y, z; // 3-D coordinates

public ThreeD() { x = y = z = 0; }

public ThreeD(int i, int j, int k) { x = i; y = j; z = k; }

// Overload binary +.

public static ThreeD operator +(ThreeD op1, ThreeD
op2)

{

```
ThreeD result = new ThreeD();  
result.x = op1.x + op2.x;  
result.y = op1.y + op2.y;  
result.z = op1.z + op2.z;  
return result;  
}
```

// This is now explicit.

```
public static explicit operator int(ThreeD op1)
{
    return op1.x * op1.y * op1.z;
}
```

// Show X, Y, Z coordinates.

```
public void Show()
{
    Console.WriteLine(x + ", " + y + ", " + z);
}
}
```

```
class ThreeDDemo {  
    static void Main() {  
        ThreeD a = new ThreeD(1, 2, 3);  
        ThreeD b = new ThreeD(10, 10, 10);  
        ThreeD c = new ThreeD();  
        int i;  
        Console.Write("Here is a: ");  
        a.Show();  
        Console.WriteLine();  
        Console.Write("Here is b: ");  
        b.Show();  
        Console.WriteLine();  
    }  
}
```

```
c = a + b; // add a and b together
Console.Write("Result of a + b: ");
c.Show();
Console.WriteLine();
i = (int) a; // explicitly convert to int -- cast
            required
Console.WriteLine("Result of i = a: " + i);
Console.WriteLine();
i = (int)a * 2 - (int)b; // casts required
Console.WriteLine("result of a * 2 - b: " + i);
}
}
```

- Because the conversion operator is now marked as explicit, conversion to **int** must be explicitly cast.
- For example, in this line:
`i = (int) a; // explicitly convert to int -- cast required`
if the cast is removed , the program will not compile.

- There are a few restrictions to conversion operators:
- Either the target type or the source type of the conversion must be the class in which the conversion is declared.
for example, we cannot redefine the conversion from **double to int**.
- We cannot define a conversion to or from **object**.
- We cannot define both an implicit and an explicit conversion for the same source and target types.
- We cannot define a conversion from a base class to a derived class.
- We cannot define a conversion from or to an interface.

- In addition to these rules, there are suggestions that we should normally follow when choosing between implicit and explicit conversion operators.
- Although convenient, implicit conversions should be used only in situations in which the conversion is inherently errorfree.
- To ensure this, implicit conversions should be created only when these two conditions are met:
 - First, that no loss of information, such as truncation, overflow, or loss of sign, occurs.
 - Second, that the conversion does not cause an exception.
- If the conversion cannot meet these two requirements, then we should use an explicit conversion.

Operator Overloading Internals

- Internally, Microsoft Intermediate Language (MSIL) does not support operator methods.]
- For that reason, the C# compiler converts operator methods into normal functions

In table, the functions that the C# compiler substitutes for mathematical and logical operators are listed

Operator	Replacement method
operator+	op_Addition
operator-	op_Subtraction
operator*	op_Multiply
operator++	op_Increment
operator--	op_Decrement
operator/	op_Division
operator%	op_Modulus
operator&	op_BitwiseAnd
operator&&	op_LogicalAnd
operator	op_BitwiseOr
operator	op_LogicalOr

Operator	Replacement method
operator^	op_ExclusiveOr
operator false	op_False
operator true	op_True
operator>>	op_RightShift
operator<<	op_LeftShift
operator!	op_LogicalNot
operator~	op_OnesComplement

Table lists the conventional functions substituted for relational operators

Operator	Replacemnt method
operator>	op_GreaterThan
operator<	op_LessThan
operator>=	op_GreaterThanOrEqual
operator<=	op_LessThanOrEqual
operator==	op_Equality
operator!=	op_Inequality

- Conversion operators are implemented as `op_Explicit` and `op_Implicit` methods.
- The following class has two explicit operators and one implicit conversion operator:

```
public class Aclass {  
    public static explicit operator int(Aclass obj) {  
        return 0;  
    }  
    public static explicit operator float(Aclass obj) {  
        return (float) 0;  
    }  
    public static implicit operator double(Aclass obj) {  
        return 0;  
    }  
}
```

Internal view of the class

- There are two overloaded op_Explicit methods and one op_Implicit method.
- Donis.CSharpBook.Aclass
 - .class public auto ansi beforefieldinit
 - .ctor : void()
 - op_Explicit : float32(class Donis.CSharpBook.Aclass)
 - op_Explicit : int32(class Donis.CSharpBook.Aclass)
 - op_Implicit : float64(class Donis.CSharpBook.Aclass)